TRAITEMENT D'IMAGE · COURS

Vision artificielle

Luminance, contraste, transformations et histogrammes — du pixel brut à l'image exploitable.

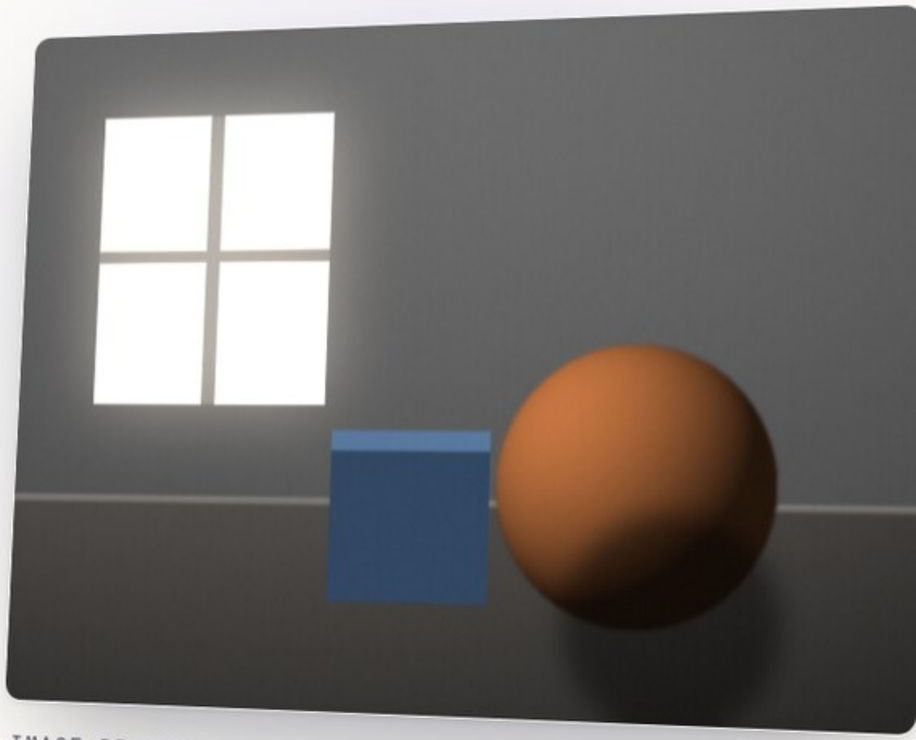


IMAGE DE TEST · 640 × 480

IMAGE DE TEST · MESURES

LUMINANCE	ÉCART-TYPE	DYNAMIQUE
93,5	16,5	67–136

LUMINANCE · CONTRASTE · TRANSFORMATIONS · HISTOGRAMMES

00 · SOMMAIRE

Quatre parties, une seule image.

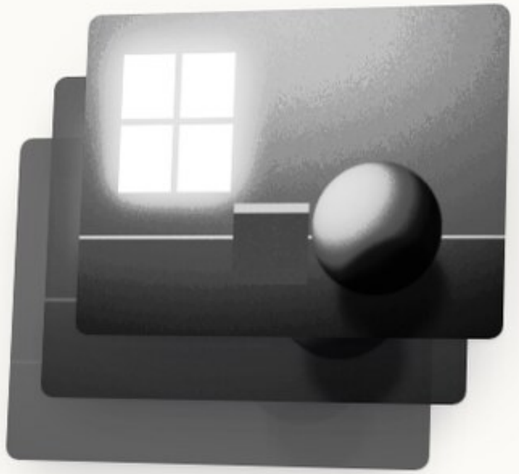
Chaque notion est illustrée sur la même photo de test.

I **Luminance & contraste** — 5 PAGES · 6 MIN
Ce que mesure un pixel, et ce que cela dit de l'image.

II **Transformations** — 4 PAGES · 7 MIN
Points, géométrie, filtrage : trois familles d'opérations.

III **Histogrammes** — 3 PAGES · 7 MIN
Lire, étirer, égaliser — la dynamique d'une image.

IV **Démonstration** — 2 PAGES · 3 MIN
NumPy et Matplotlib, sans bibliothèque de traitement d'image.

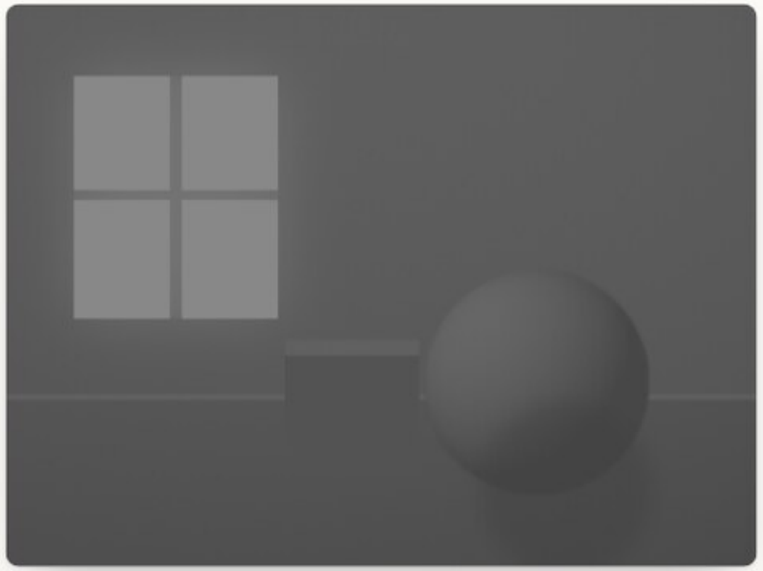


ORIGINALE → ÉTIRÉE → ÉGALISÉE

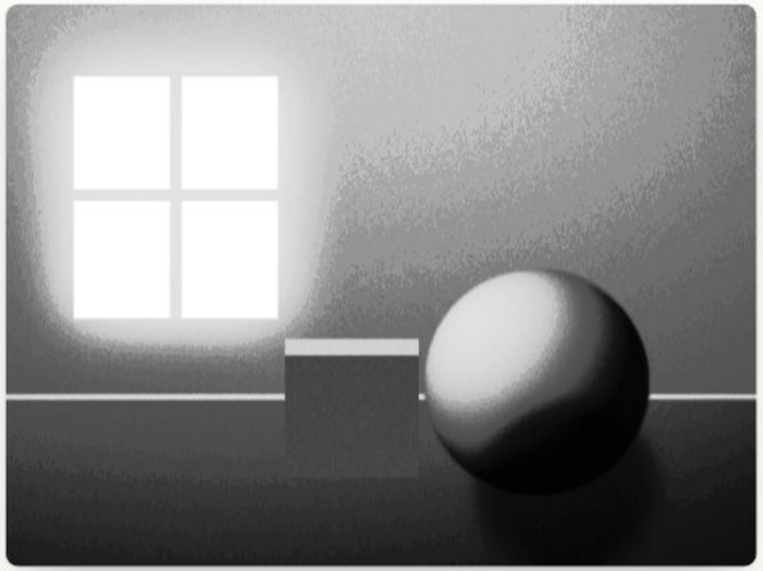
Avant de détecter, il faut voir.

Détection, segmentation, reconnaissance : tout commence par une image exploitable.

BRUT · σ 16,5



ÉGALISÉ · σ 75,9



Un capteur ne rend pas ce que l'œil perçoit.

LUMINANCE

trop sombre, trop claire

CONTRASTE

niveaux tassés en gris moyen

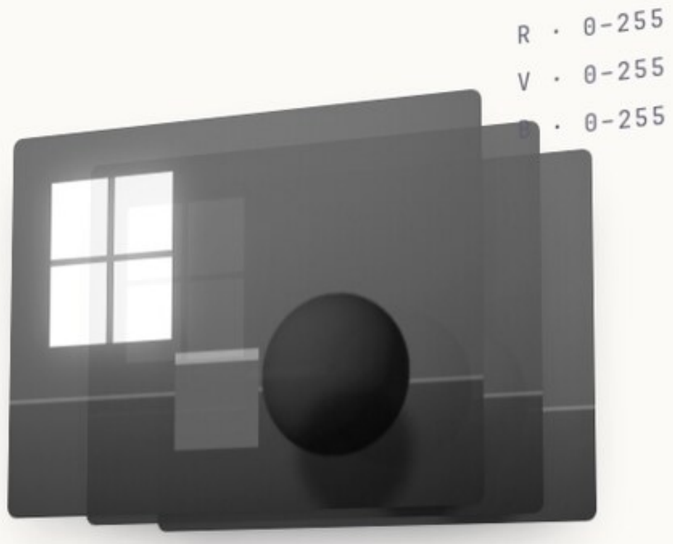
BRUIT

grain et impulsions

Le prétraitement ne crée pas d'information : il la rend lisible.

Une image est une matrice.

M × N pixels, chacun décrit par trois canaux.



MATRICE

640 × 480

une grille de valeurs, pas un objet continu

QUANTIFICATION

0 → 255

8 bits par canal : 256 niveaux par pixel

CANAUX

R, V, B

le gris se calcule, il ne se devine pas

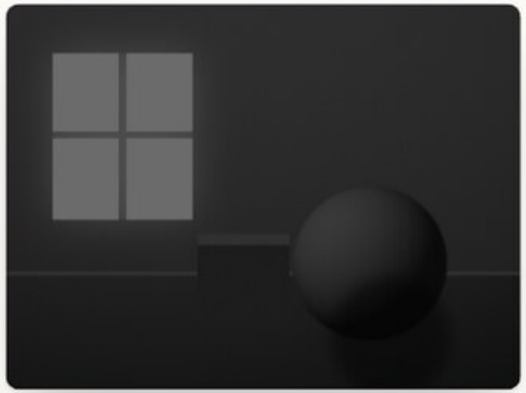
$$Y = 0{,}299 \cdot R + 0{,}587 \cdot V + 0{,}114 \cdot B$$

→ page suivante

Quatre images, quatre diagnostics.

Même scène, même cadrage : seules la position et la largeur du signal changent.

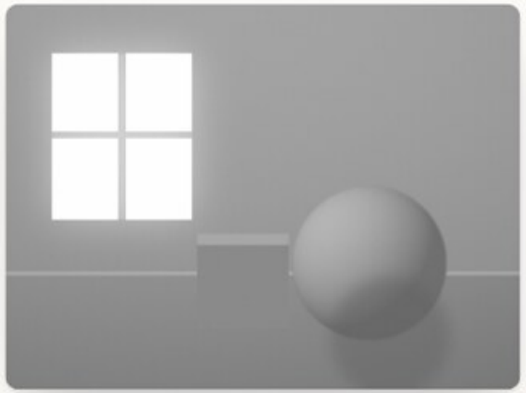
SOMBRE



Ȳ 43,9 σ 24,5



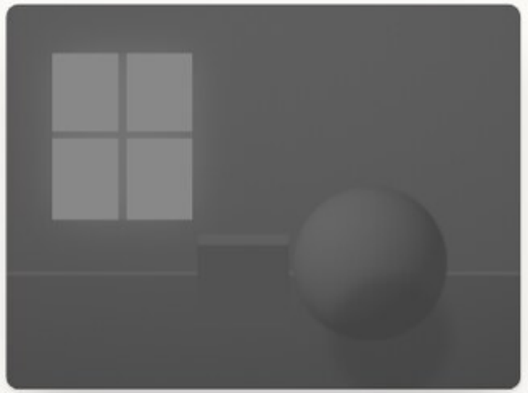
CLAIRE



Ȳ 160,9 σ 36,2



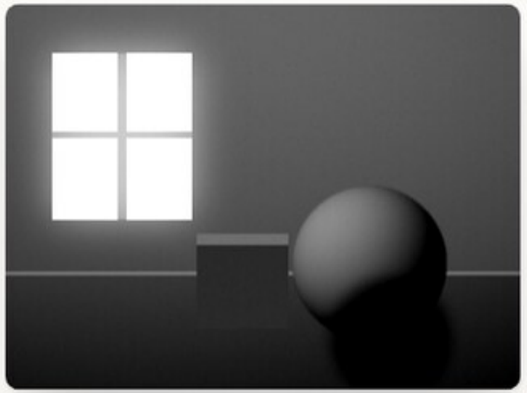
FAIBLE CONTRASTE



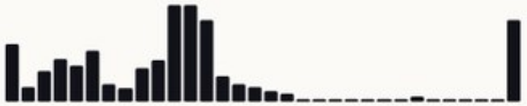
Ȳ 93,5 σ 16,5



FORT CONTRASTE



Ȳ 89,3 σ 67,9



La **position** dit l'exposition ; la **largeur** dit le contraste. Les deux se lisent d'un coup d'œil.

La luminance, c'est la brillance perçue.

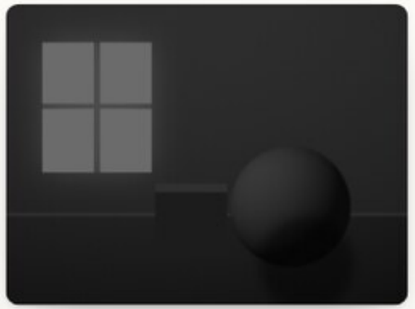
Une somme pondérée des canaux : l'œil n'est pas également sensible au rouge, au vert et au bleu.

≈ 59 % de la perception

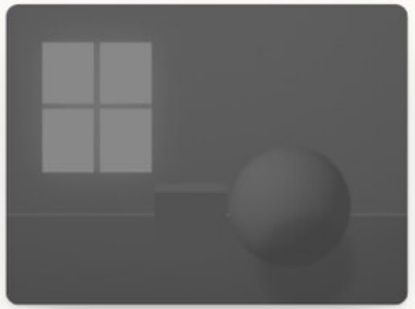
LUMINANCE · CANAL Y

$$Y = 0{,}299 \cdot R + 0{,}587 \cdot V + 0{,}114 \cdot B$$

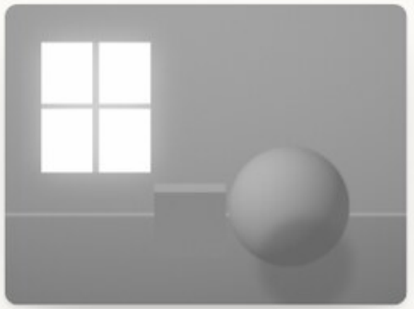
SOMBRE · $\bar{Y}$ 43,9



RÉFÉRENCE · $\bar{Y}$ 93,5



CLAIRE · $\bar{Y}$ 160,9



La **luminance moyenne** résume l'exposition globale. Elle ne dit rien de la répartition — c'est le rôle de l'histogramme.

Le contraste, c'est l'écart.

Trois façons de le mesurer — et ce que chacune laisse de côté.

ÉCART-TYPE

σ = 16,5

$$\sigma = \sqrt{\sum (r_i - \bar{Y})^2 / N}$$

La dispersion réelle des niveaux. La mesure la plus robuste.

MICHELSON

0,34

$$(I_{max} - I_{min}) / (I_{max} + I_{min})$$

Ne regarde que deux pixels : ici la fenêtre et l'ombre.

DYNAMIQUE

67 → 136

$$I_{max} - I_{min} = 69$$

Dit si l'image sature — et de combien on peut l'étaler.

Sur la même image, deux mesures racontent deux choses différentes : σ décrit la scène, Michelson décrit ses extrêmes.

Trois façons de transformer une image.

Tout ce qui suit entre dans l'une de ces trois familles.

01



Ponctuelles

Un pixel d'entrée donne un pixel de sortie. Le voisinage n'existe pas.

$s = f(r)$

02



Géométriques

On déplace les pixels dans le plan : translation, rotation, échelle.

$(x, y) \to (x', y')$

03



Filtrage

Un pixel devient une combinaison de ses voisins, via un noyau.

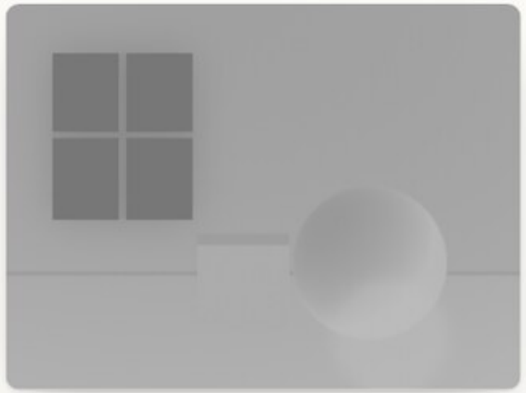
$s = \Sigma\, w_i \cdot r_i$

Chaque pixel, indépendamment.

s = f(r) : le voisinage n'entre jamais dans le calcul.

Négatif

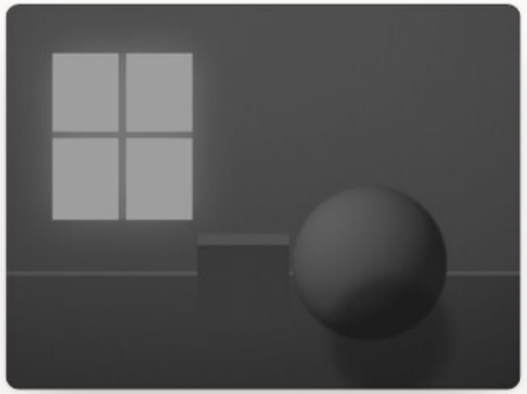
`s = 255 - r`



Inverse les niveaux. Utile pour lire une densité.

Linéaire

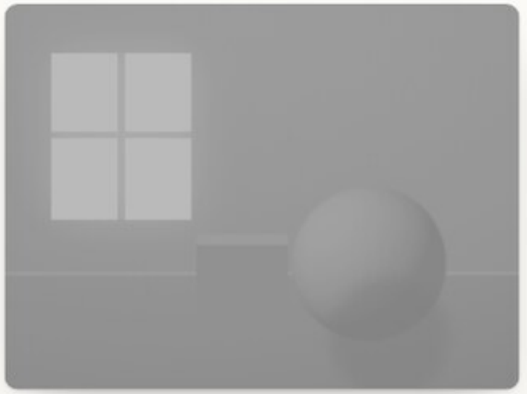
`s = a·r + b`



a règle le contraste, b la luminosité.

Gamma

`s = c · r^γ`



γ < 1 éclaircit les ombres, γ > 1 les ferme.

Seuillage

`s = 0 ou 255`



Binarise : on perd le détail, on gagne la mesure.

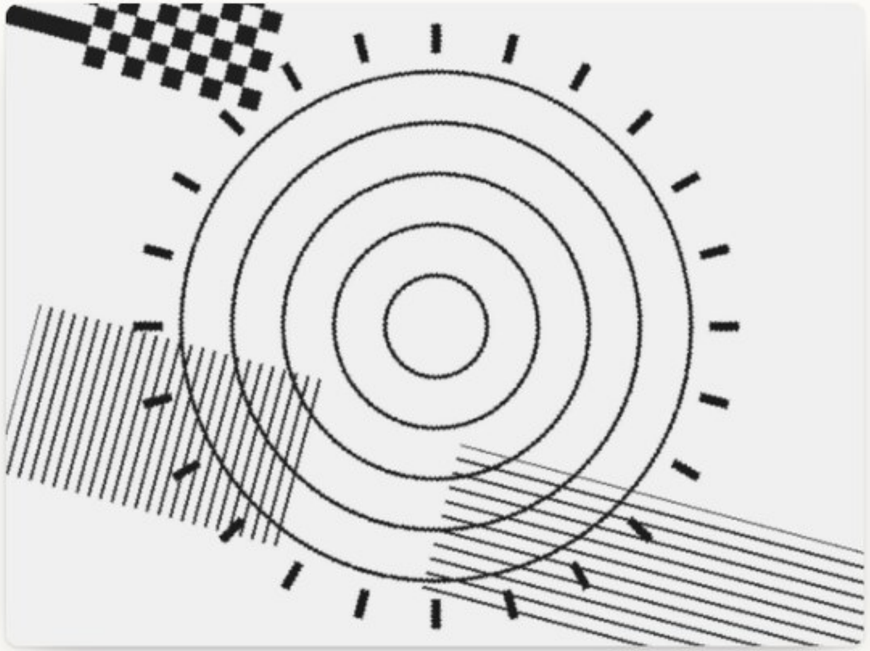
Un seul réglage suffit à changer toute l'image — d'où l'importance de le choisir sur des mesures, pas à l'œil.

Déplacer les pixels ne suffit pas.

Rotation de 15° : selon l'interpolation, le résultat crispe ou lisse.

Plus proche voisin

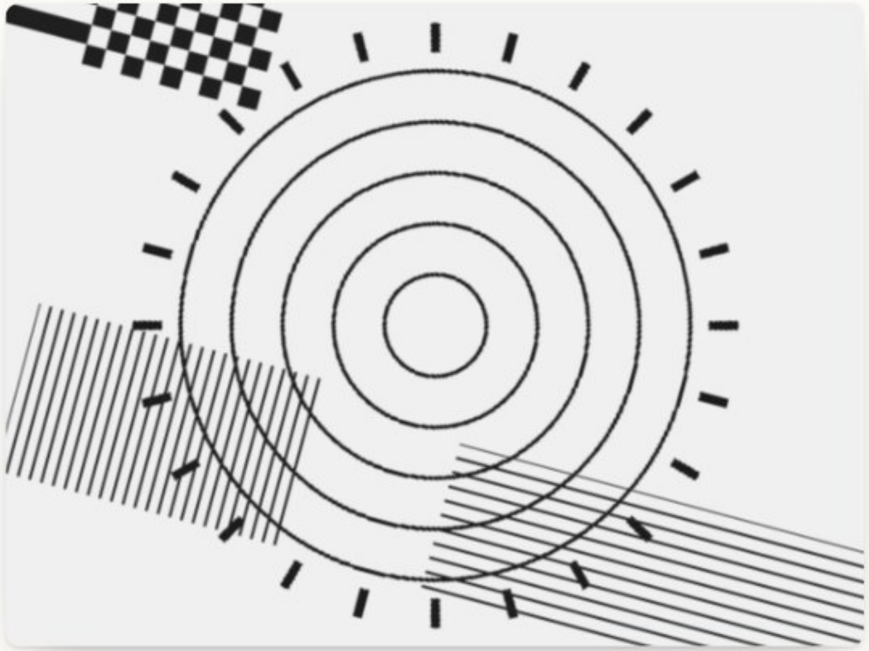
mire de test



Un pixel source par pixel cible → escaliers visibles.

Bilinéaire

mire de test



Moyenne pondérée des 4 voisins → contours plus doux.

$$I(x,y) \approx (1-fx)(1-fy)\cdot I_{00} + fx(1-fy)\cdot I_{10} + (1-fx)fy\cdot I_{01} + fx\cdot fy\cdot I_{11}$$

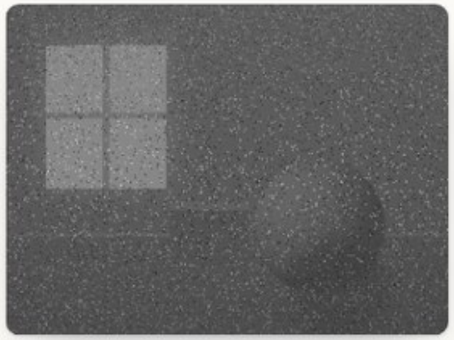
Un pixel vaut son voisinage.

Convoluer, c'est remplacer chaque pixel par une combinaison de ses voisins.

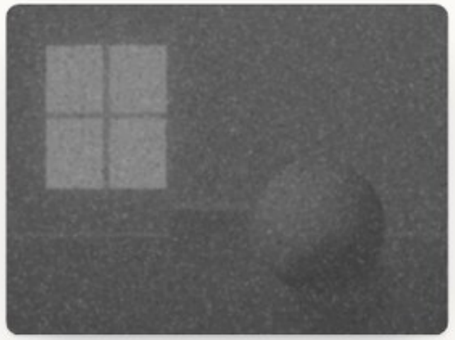
1	1	1
1	1	1
1	1	1

MOYENNE · ÷ 9

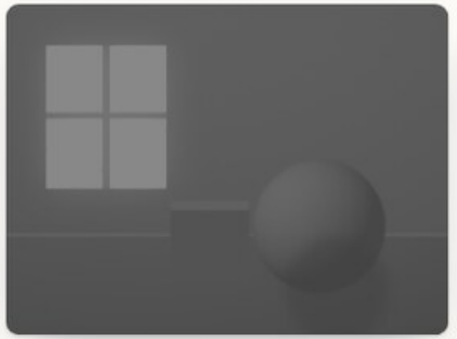
Neuf coefficients, une somme pondérée : c'est tout un filtre linéaire. Changer le noyau change l'effet — `moyenne` lisse, `Sobel` dérive, un noyau gaussien pèse le centre plus fort que les bords.



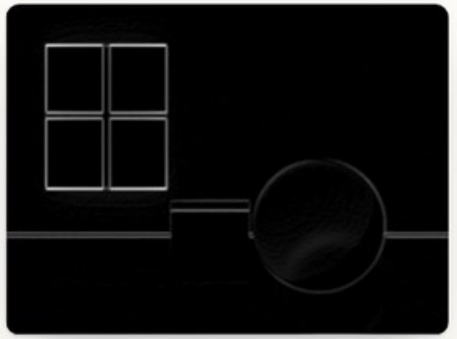
Bruit
Impulsions et grain.



Moyenne
Lisse, mais étale le bruit.



Médiane
Impulsions supprimées.



Sobel
Contours et gradients.

L'histogramme compte, il ne localise pas.

Le nombre de pixels par niveau — jamais leur position dans l'image.

$$h(k) = \#\{ \text{pixels d'intensité } k \}$$

h(k) : combien de pixels valent exactement k

$$p(k) = h(k) / N$$

p(k) : la même chose, en proportion du total

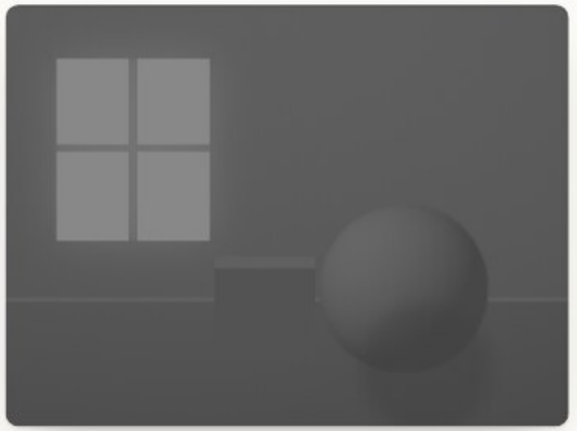
$$C(k) = \Sigma_{i \le k}\ h(i)$$

C(k) : le cumul — c'est lui qui servira à égaliser

`position` → exposition globale

`largeur` → contraste

`pics` → grandes régions uniformes



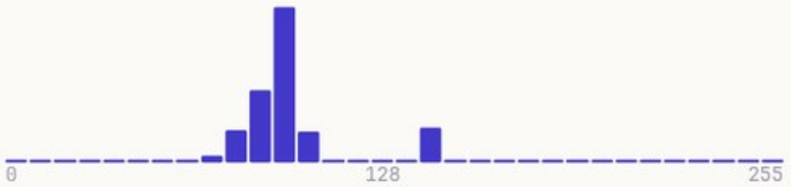
MOYENNE

93,5

ÉCART-TYPE

16,5

67 → 136
aucun pixel saturé



70 / 256

niveaux utilisés : l'histogramme est étroit, donc le contraste est faible.

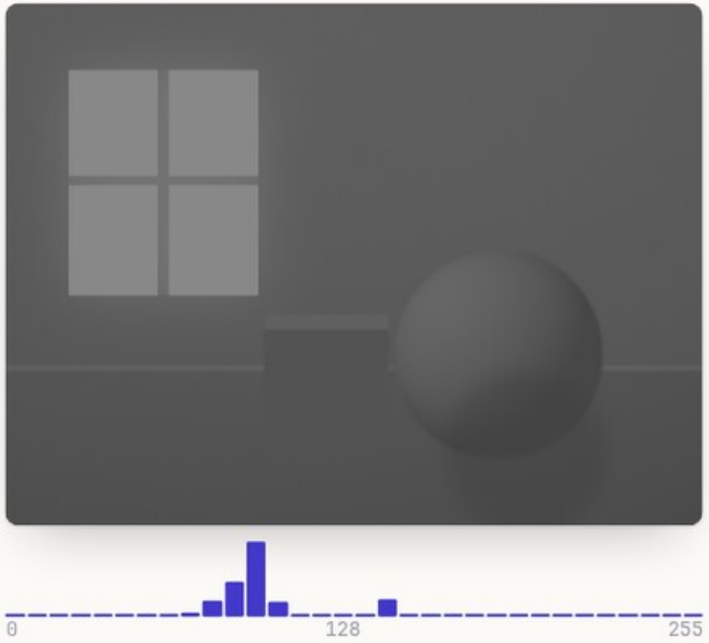
Étaler la dynamique.

Une transformation linéaire qui remet rmin à 0 et rmax à 255.

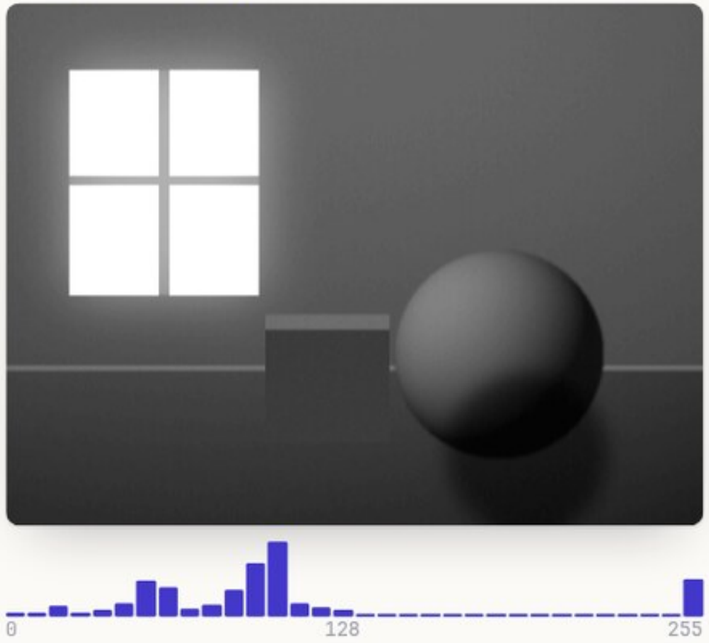
$$s = 255 \cdot (r - r_{min}) / (r_{max} - r_{min})$$

rmin = 67 rmax = 136

AVANT · σ 16,5 67–136



APRÈS · σ 60,9 0–255



Égaliser, c'est redistribuer.

La CDF devient la table de correspondance : chaque niveau reçoit sa part de pixels.

1 **Compter**

`h = histogramme de l'image`

2 **Cumuler**

`C = cumsum(h)`

3 **Normaliser**

$LUT(k) = 255 \cdot (C(k) - C_0) / (N - C_0)$

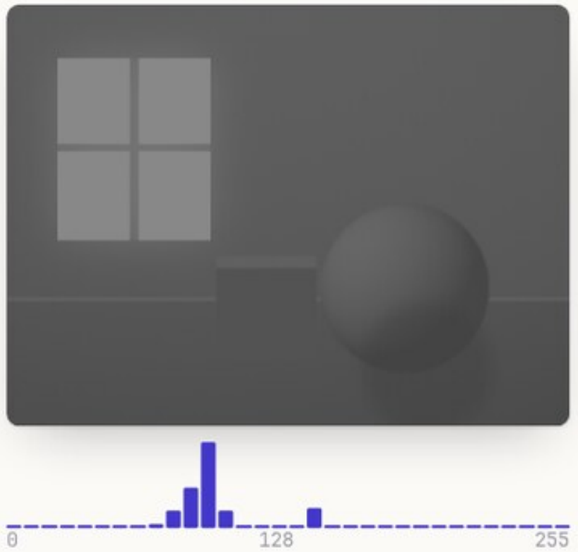
4 **Appliquer**

`s = LUT[r]` — une simple table

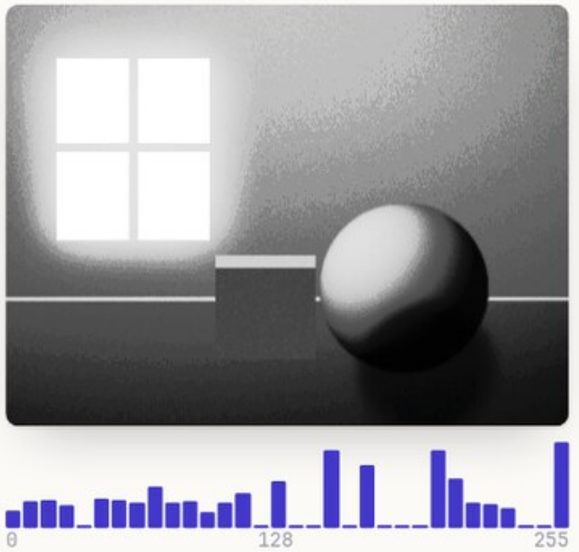
Limites : elle amplifie le grain et peut paraître artificielle.

`aller plus loin` · spécification (imposer un histogramme cible) · CLAHE (égalisation locale)

AVANT



APRÈS



Tout tient en une page de NumPy.

Aucune bibliothèque de traitement d'image — seulement NumPy et Matplotlib.

1 · CHARGER, CONVERTIR

```python
import numpy as np, matplotlib.pyplot as plt

def load_gray(path):
    img = plt.imread(path)
    img = img * 255 if img.dtype != np.uint8 else img
    Y = np.dot(img[..., :3], [0.299, 0.587, 0.114])
    return np.clip(np.round(Y), 0, 255).astype(np.uint8)

# Ȳ = img.mean() · σ = img.std()
# h = np.bincount(img.ravel(), minlength=256)
```

2 · ÉTIRER, ÉGALISER

```python
def stretch(im):
    lo, hi = im.min(), im.max()
    s = 255 * (im - lo) / (hi - lo)
    return np.round(s).astype(np.uint8)

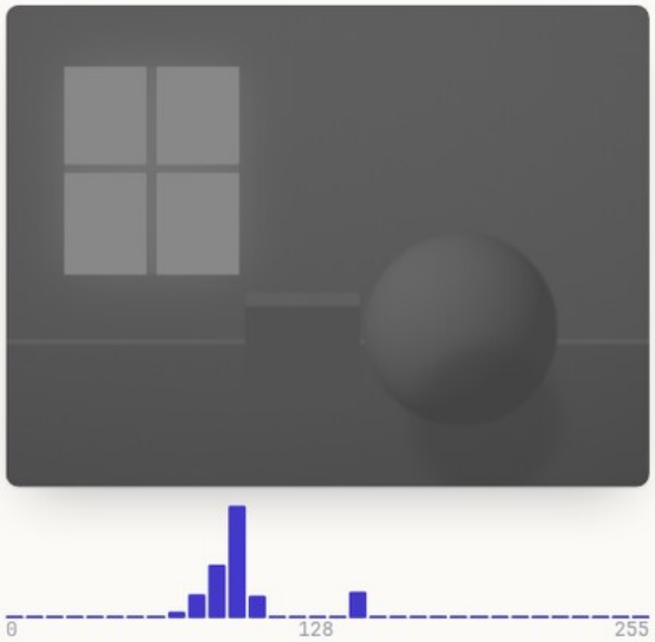
def equalize(im):
    hist = np.bincount(im.ravel(), minlength=256)
    cdf = np.cumsum(hist)
    c0 = cdf[cdf > 0].min()
    lut = np.round(255 * (cdf - c0) / (cdf[-1] - c0))
    return lut[im].astype(np.uint8)
```

$ python demo.py · le même script produit les figures de la page suivante

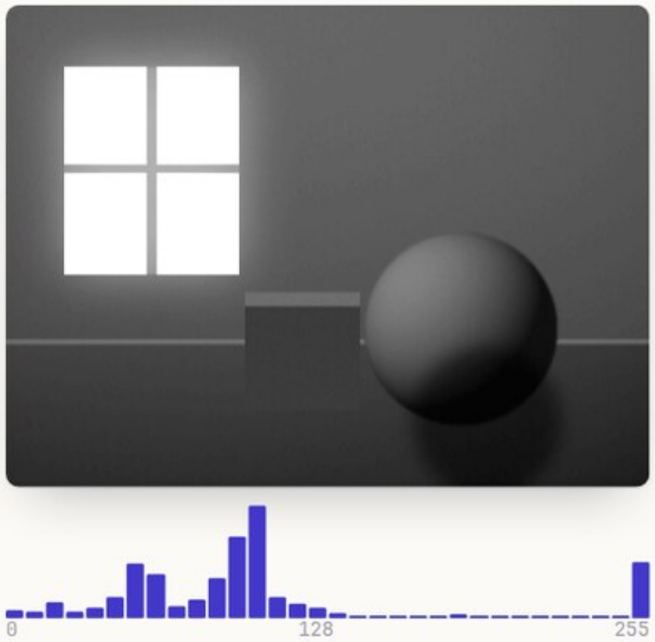
Une image, trois lectures.

Même scène, trois traitements — et les histogrammes qui les expliquent.

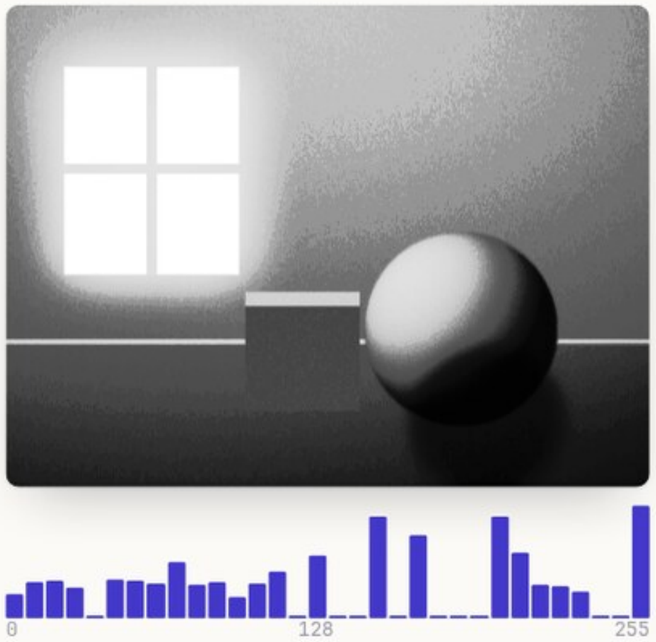
ORIGINALE — $\bar{Y}$ 93,5 · σ 16,5



ÉTIRÉE — $\bar{Y}$ 97,9 · σ 60,9



ÉGALISÉE — $\bar{Y}$ 134,2 · σ 75,9



Merci — place à la démonstration.

```
$ python demo.py
```